

CS:5980

Foundations of Embedded Systems

Safety Requirements

Part III

Copyright 2014-26, Rajeev Alur and Cesare Tinelli.

Created by Cesare Tinelli at the University of Iowa from notes originally developed by Rajeev Alur at the University of Pennsylvania. These notes are copyrighted materials and may not be used in other course settings outside of the University of Iowa in their current form or modified form without the express written permission of one of the copyright holders. During this course, students are prohibited from selling notes to or being paid for taking notes by any person or commercial firm without the express written permission of one of the copyright holders.

Requirements-based Design

Systematic approach to design of systems

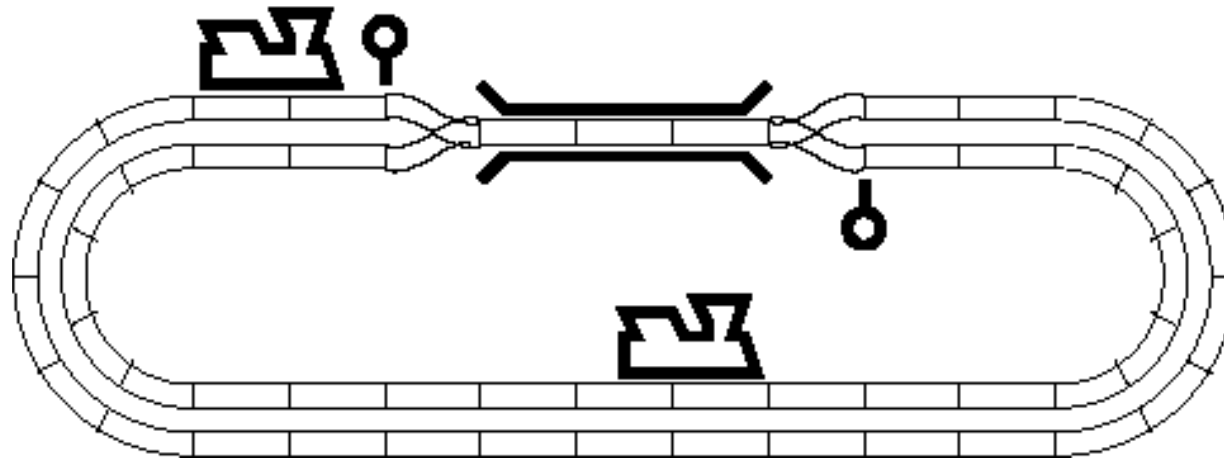
Given:

- Input/output interface of system/component C to be designed
- Model E of the environment
- Safety properties $P_1, \dots, P_n$ of the composite system

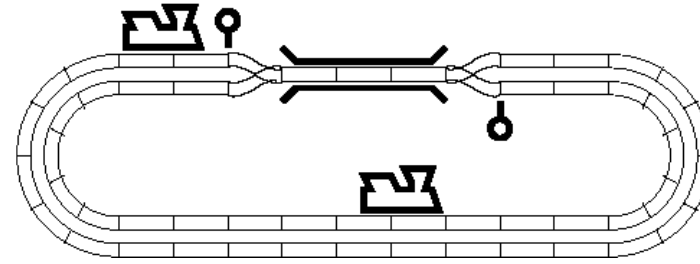
Design problem:

- Fill in details of C (state variables, initialization, and update) so that $P_1, \dots, P_n$ are invariant for $C \parallel E$

Railroad Controller Example

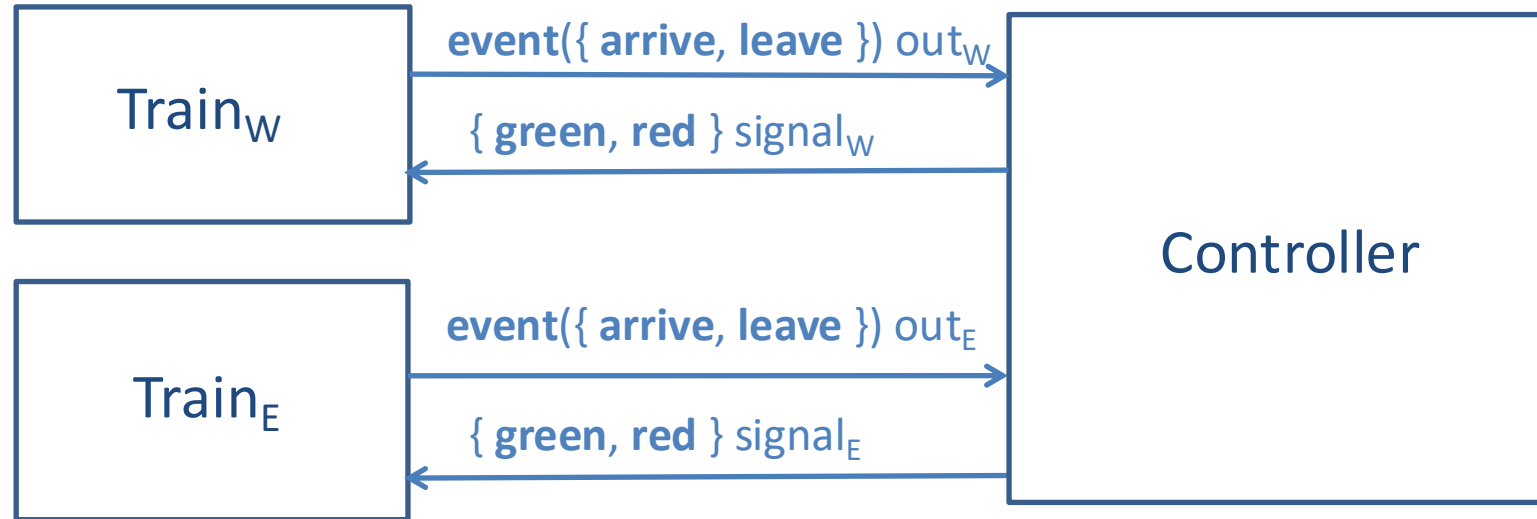


Train Model



- Each train is initially away from bridge
- Train can be in **away** state for an arbitrarily long period
- When the train gets close, it communicates with the traffic controller via an event, say, **arrive**, and moves to a different state, say, **wait**
- When near, train is monitoring the signal on the bridge:
 - If the signal is **green**, it enters the bridge
 - If the signal is **red**, it continues to wait
- A train can stay on bridge for a duration that is not exactly known (and not directly under the control of the traffic controller)
- When the train leaves the bridge, it communicates with the controller via an event, say, **leave**, and goes back to **away** state
- This behavior repeats: an away train may again request entry
- The two trains have symmetric behavior

Controller Design Problem

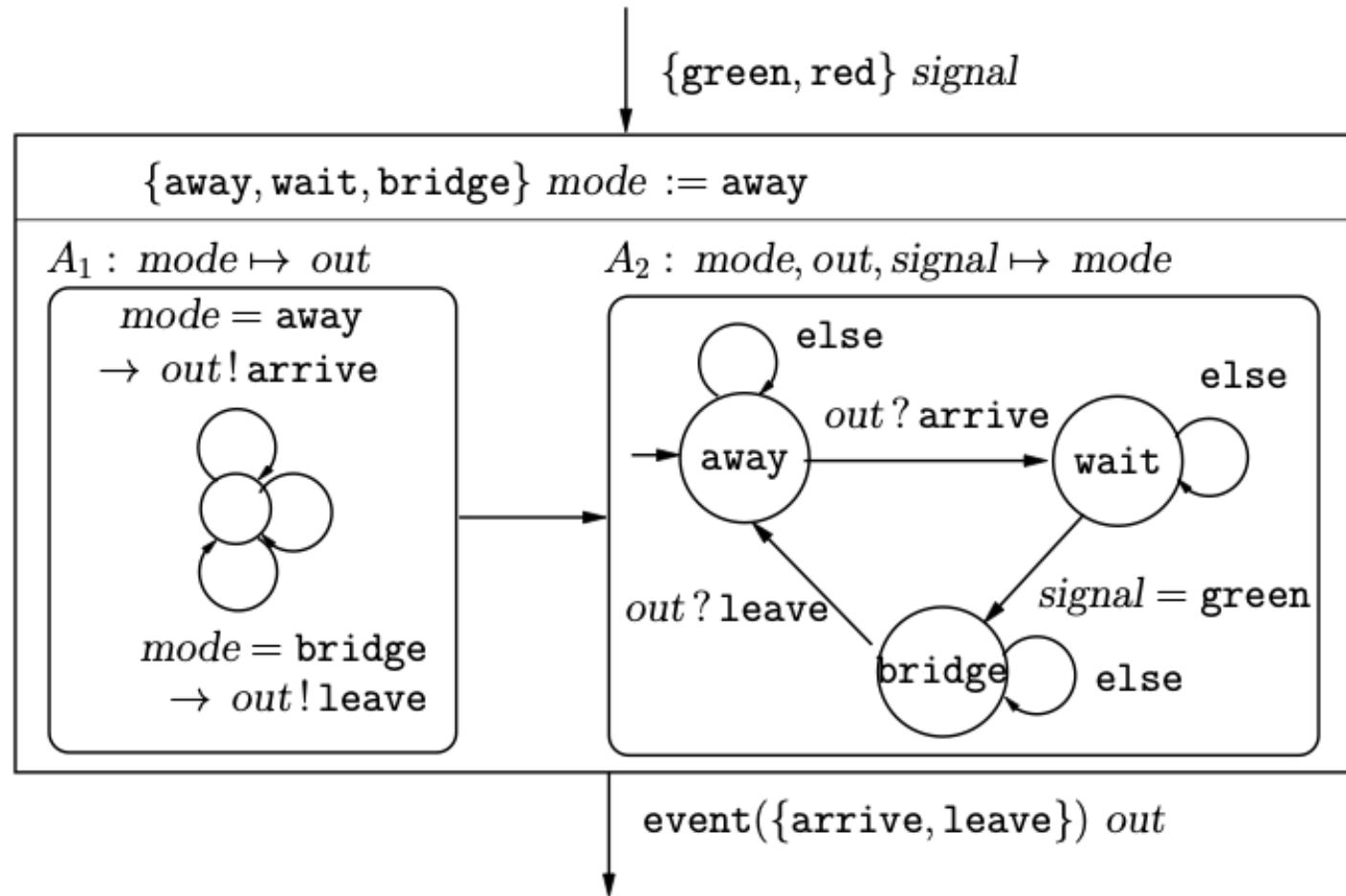


Safety Requirement: Trains should not be on bridge simultaneously

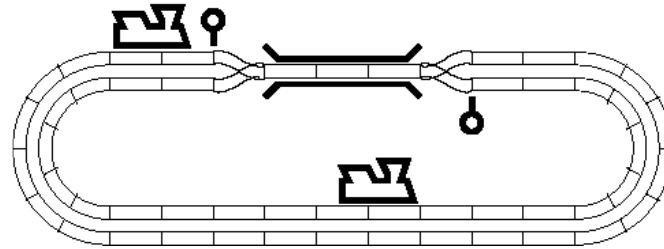
Formally, the following should be an **invariant** of the system:

$$\neg(\text{mode}_W = \text{bridge} \wedge \text{mode}_E = \text{bridge})$$

Synchronous Component Train

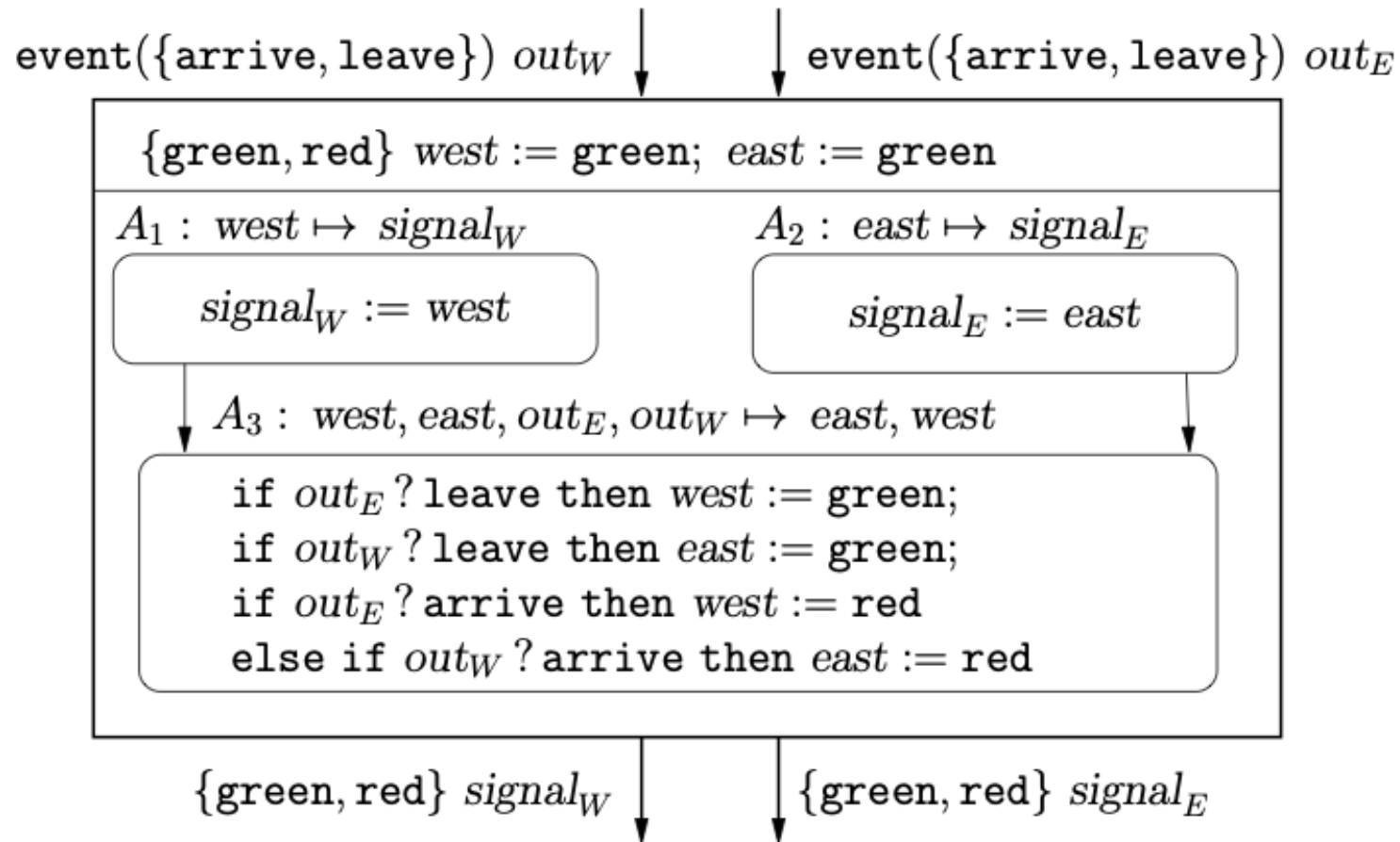


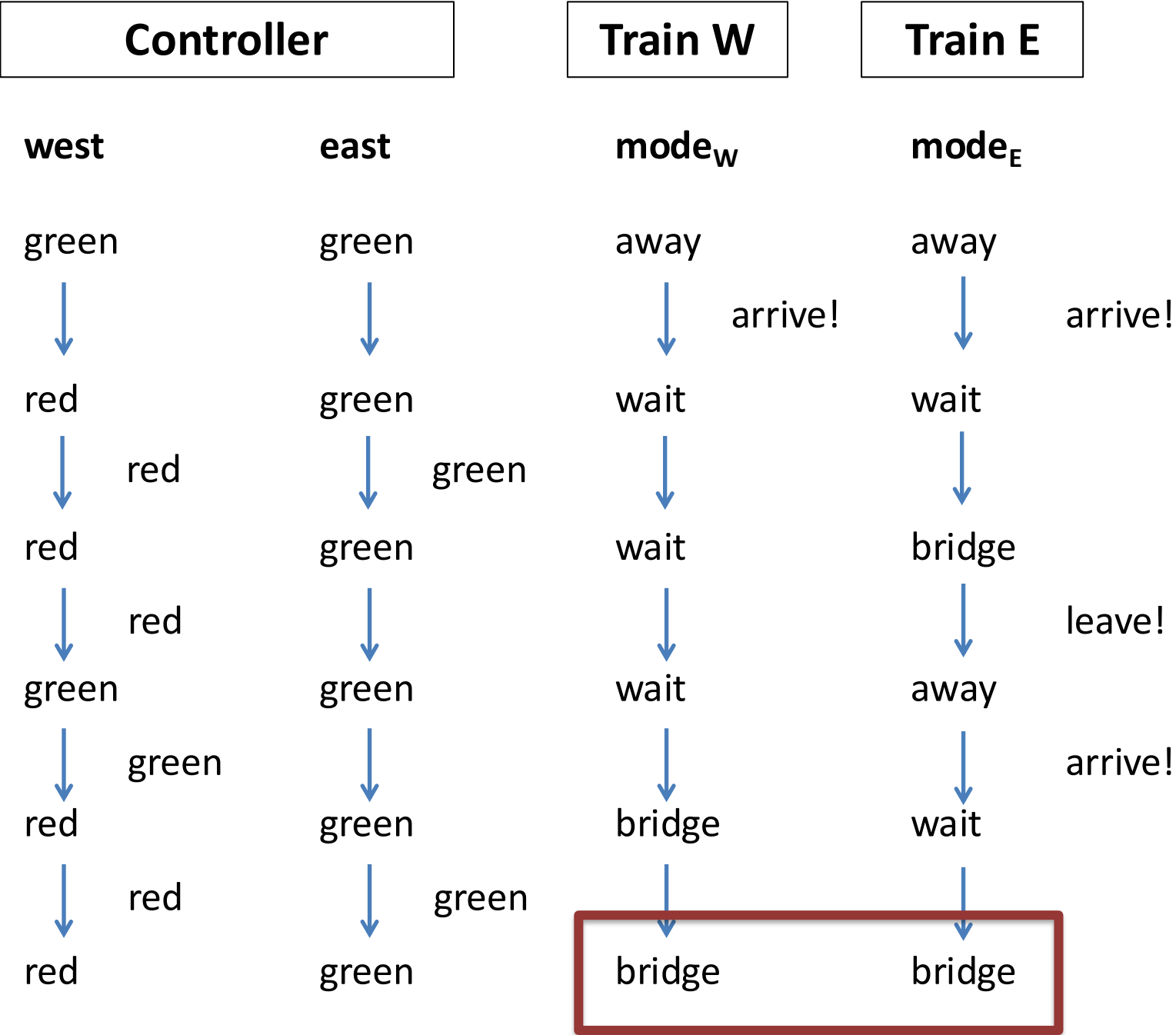
First Attempt at Controller Design



- Controller maintains state variables **east**, **west** to track the state of each signal
- Both state variables are initially **green**
- The output for the signals is based on the corresponding state vars
- When a train arrives, update the opposite signal var to **red** to block the other train from entering the bridge
- When a train leaves, reset the opposite signal var to **green**
- What happens if both trains arrive simultaneously?
 - Give priority to east train: set **west** signal var to **red**

Synchronous Component Controller1

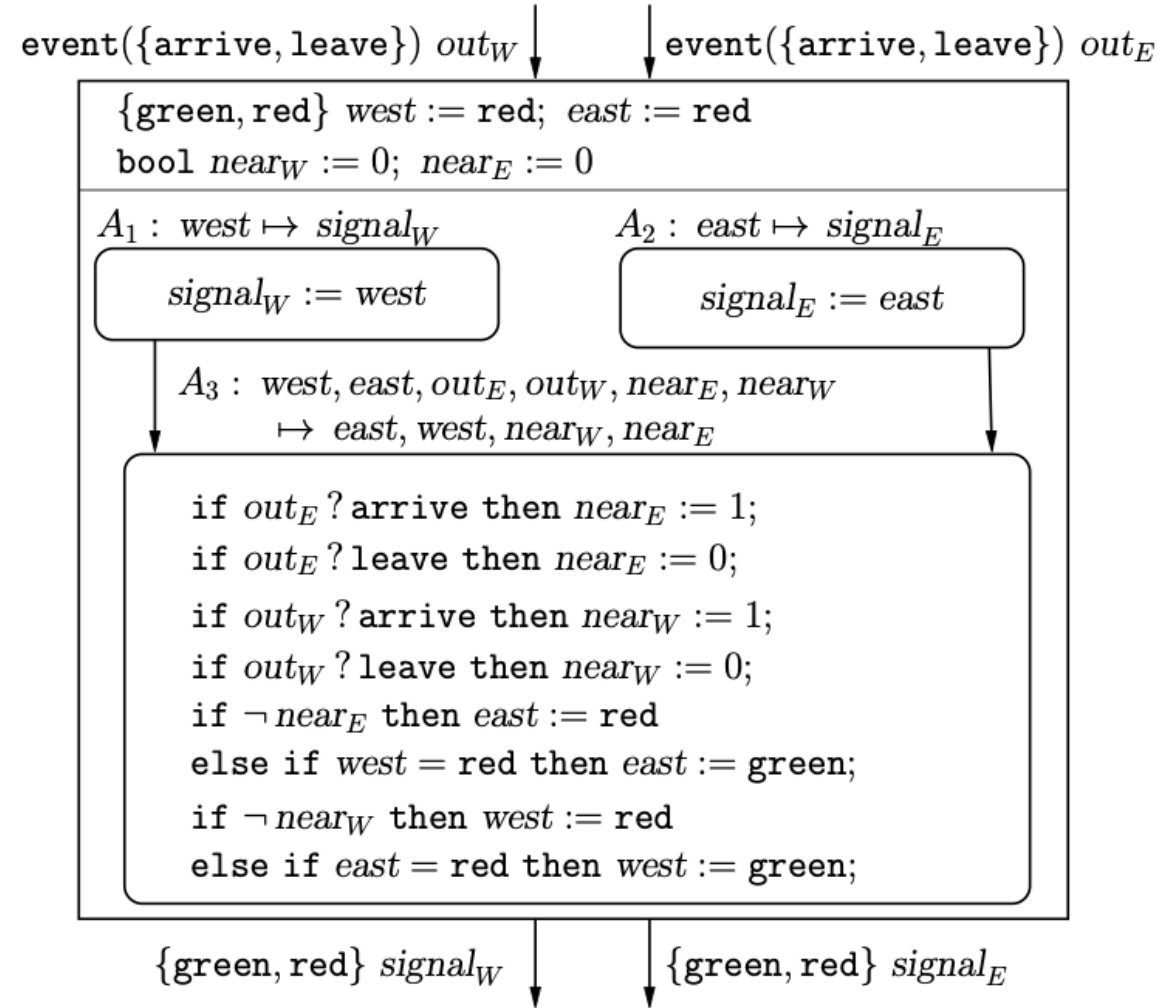




Second Attempt at Controller Design

- What went wrong the first time?
 - Controller did not remember whether a train was waiting at each entrance
- Boolean variable near_w remembers if the west train wants to use the bridge
 - Initially 0
 - When the west train issues arrive , changed to 1
 - When the west train issues leave , reset back to 0
- Invariant: $\text{mode}_w = \text{away} \Leftrightarrow \text{near}_w = 0$
- Variable near_e is symmetric
- Let's also start with both signals red
- A signal is changed to green if the corresponding train is near and the other signal is red ; it is changed back to red when train is away
- Need still to resolve simultaneous arrivals by preferring one train

Second Attempt at Controller Design



Properties of Controller2

- $\text{RailRoadSystem2} = \text{Controller2} \parallel \text{Train}_W \parallel \text{Train}_E$ satisfies the safety property
 $\neg(\text{mode}_W = \text{bridge} \wedge \text{mode}_E = \text{bridge})$
- What about some additional properties?
 1. *If the west train is waiting, then west signal will eventually become green*
 2. *If the west train is waiting, other train should not be allowed on bridge more than once*
- Requirement 1 is a **liveness** requirement (see Chap. 5 of text)
- Requirement 2 is a **safety** requirement
 - Its violation can be demonstrated by a (finite) execution in which east train enters, leaves, and enters again while west train keeps waiting with its signal red
 - But it cannot be encoded as an invariant on system state vars!

Safety Monitor

Monitor M for a system observes its inputs/outputs, and enters an error state if undesirable behavior is detected

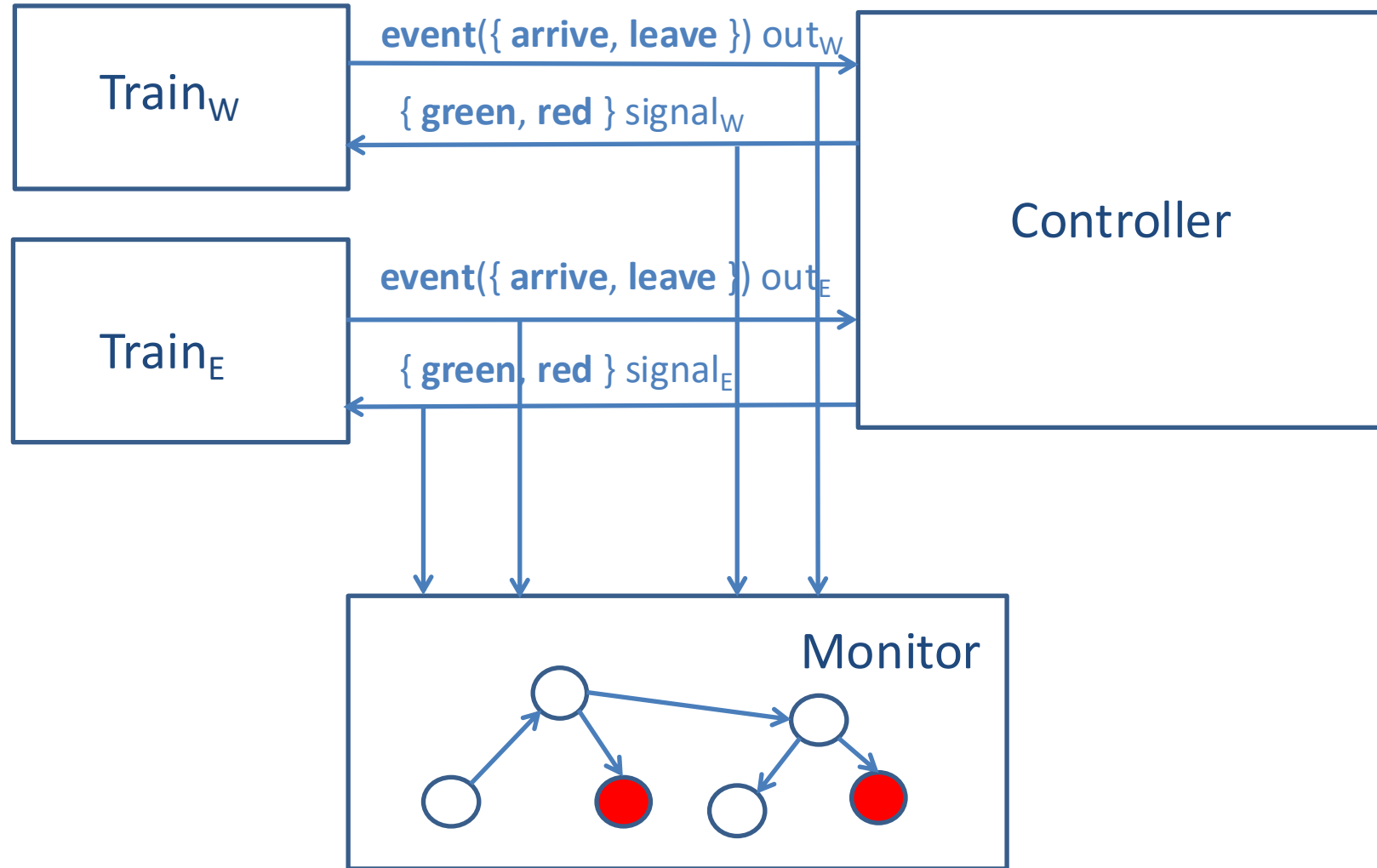
Monitor M is specified as extended state machine

1. The set of input variables of M = input/output variables of system being monitored
2. An output of M cannot be an input to system
(monitor does not influence what the system does)
3. A subset F of modes of state-machine declared as accepting

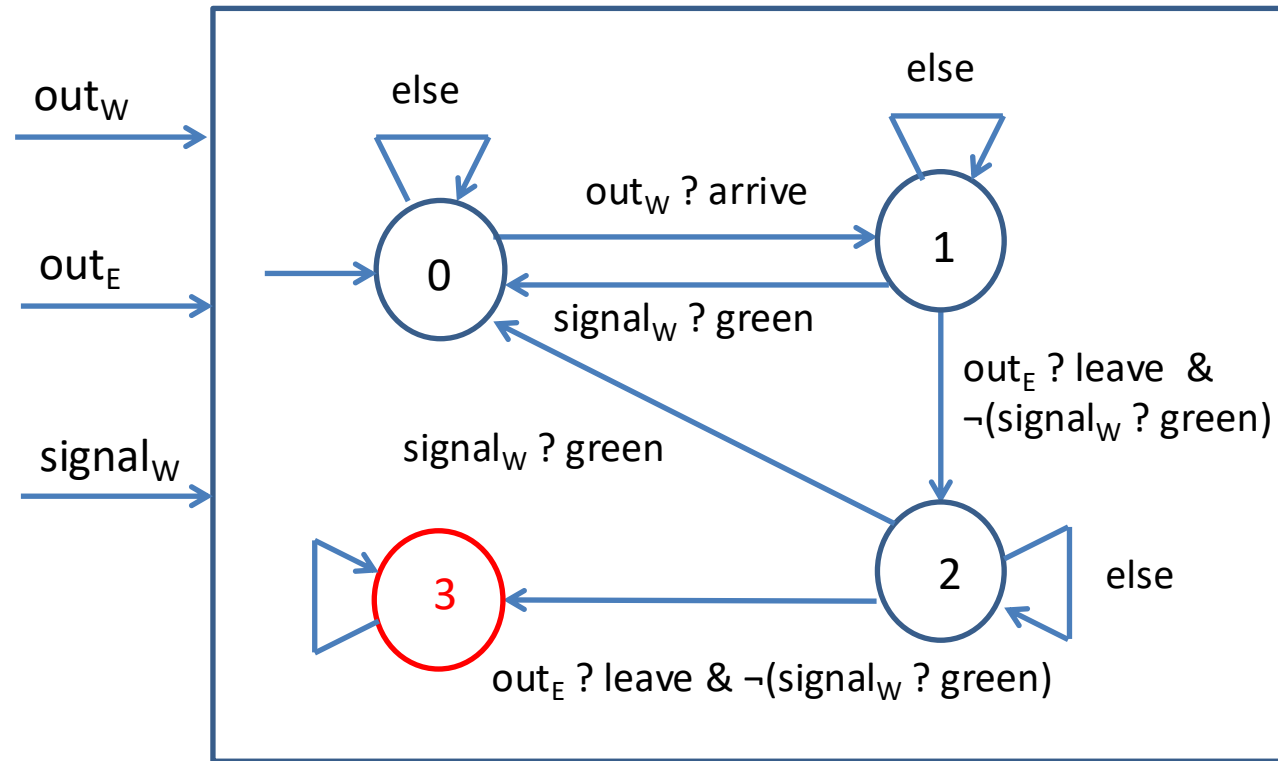
Undesirable behavior: An execution that leads monitor state to F

Safety verification: Check whether ($M.mode$ not in F) is an invariant of $C \parallel M$

Safety Monitors



Monitor to check fairness for railroad



Error execution: As west train waits, east train is allowed on bridge twice

Exercise: Leader Election

Suppose we want to check that at most one of the nodes declares itself to be the leader

1. Design a monitor M

- Input variables: $\{\text{undecided}, \text{leader}, \text{follower}\} \text{ status}_n$, for each node n
- M should enter error state iff for two distinct nodes m and n
 - there exists a round r_1 in which $\text{status}_m = \text{leader}$ and
 - there exists a round r_2 in which $\text{status}_n = \text{leader}$

2. Consider the requirement: eventually $\text{status}_n \neq \text{undecided}$

Why can't we design a monitor that enters an error state if this requirement is violated?

Credits

Notes based on Chapter 3 of

Principles of Cyber-Physical Systems

by Rajeev Alur

MIT Press, 2015